COMP-8117 · SPLASH PROJECT PLAN

The "Success Disaster"

Scaling in-house GenAI inference through a launch-day spike without paying for idle GPUs

Team: *[names]* **Duration:** 10 weeks **Defence:** December 2026 **Version:** final 1.0

01 Problem

Startups running their own models (fine-tuned LLMs, vision models, RAG) usually rent dedicated GPU machines. One always-on NVIDIA A10G (AWS `g5.xlarge`) costs about **$1.01/hour ≈ $735/month**. Provisioning enough of them for a launch peak (say 8 GPUs) costs **≈ $5,900/month**, even though average usage may need only one. Even the cheapest AWS GPU, a T4 (`g4dn.xlarge`, $0.526/hour), costs **≈ $384/month** left running, and it still can't absorb a spike. On launch day, traffic can jump from near zero to tens of thousands of users. Under-provision and the launch crashes. Over-provision and the company burns its runway.

> **Problem statement.** How can a bootstrapped startup serve its own GenAI model through an unpredictable traffic spike, with no dropped requests and no cost when idle?

Impact. The company pays only for GPU seconds used and survives a viral launch. Users get queued, predictable responses instead of "server overloaded" errors.

02 How we run the project: a simulated DevOps team

We run the project as the platform / DevOps team of a fictional seed-stage startup, **"LaunchCo"**. The product team is shipping one AI feature *[e.g. image captioning]* on launch day. Our job is to make sure it survives the launch within budget. Everything we build, measure and present is framed as the decisions a real infra team would have to make and defend.

Constraint	Value
Launch day (deadline)	Our December defence: the live demo is the launch
Team	4 part-time engineers, 10 weeks (see §07)
Budget	**Hard cap: $1,000/month.** Planned spend is ≈ $640 over the whole project, for real staging + prod environments and GPU load tests. Credits first; we ask only for the gap (see §06).
Non-negotiables	No dropped requests · $0 GPU cost when idle · every change goes dev → staging → prod

03 Traffic projection

We can't generate 100k real users, so we model LaunchCo's expected launch curve and replay it at a **1:100 scale** with Locust. The architecture must behave the same at both scales. What changes is the number of workers and the bill, and we extrapolate both from measured cost per request.

Phase	Real-world projection	Our simulated load	What we expect the system to do
Pre-launch	~0–50 users / day	0–2 concurrent	GPU workers at 0; API on minimum size
Launch spike (first hour)	0 → 100,000 users	0 → 1,000 concurrent in ~5 min	Queue absorbs burst; workers scale to cap
Sustained peak	~20,000 active users	~200 concurrent for 20 min	Queue stays short; steady worker count
Cool-down	Back to low hundreds	200 → 0 over 10 min	Workers scale back to 0

Assumptions to confirm in week 3: requests per user, average inference time per request, and max concurrent GPU workers allowed by the provider's free tier.

Success criteria (measured, not claimed)

Criterion	Target	Evidence
Dropped / failed requests	0% (or every failure explained)	Locust report
Idle GPU cost	$0	Worker count = 0 + billing screenshot
p95 time to result during spike	*[X] s*	Dashboard
Cost per 1,000 requests	Measured	Provider bill ÷ requests served
vs. always-on GPU at our traffic curve	Cheaper, or we explain the break-even point	Same test against the baseline

04 Architecture

Load / users Locust replays the traffic curve → **API gateway** Stateless FastAPI container. Accepts, enqueues, returns job ID → **Queue** Redis. A spike becomes a backlog, not a crash → **GPU workers** Serverless GPU (RunPod / Modal). Scale with queue depth, down to 0 → **Results** Client polls by job ID

Observability runs across all of it (Grafana): queue depth, active workers, latency and spend over time. **Baseline**: the same model on one always-on GPU with no queue, used only for comparison.

05 Environments: dev, staging, prod

Each environment has one job. Code only moves forward through them, and the GPU tier is the only thing that costs money, so it is switched off wherever it isn't needed.

Dev

- **Purpose:** build and debug
- **Runs:** local Docker Compose (API + Redis)
- **Model:** CPU mock that returns a fake result, so no GPU spend
- **Deploys:** on every push to a feature branch
- **Cost:** $0

Staging

- **Purpose:** prove it before prod; **all load tests run here**
- **Runs:** identical to prod (same 10-worker cap)
- **Model:** real model on serverless GPU
- **Deploys:** on merge to `main`
- **Cost:** only during test runs

Prod

- **Purpose:** launch day (the defence demo)
- **Runs:** 10-worker cap, alerts on
- **Model:** real model, scale-to-zero
- **Deploys:** manual promotion of a tagged release that passed staging
- **Cost:** ~$0 idle; spend only during demo

Promotion rule: nothing reaches prod without passing the staging load test. **Rollback:** redeploy the previous tagged image in one command, rehearsed in week 9.

06 Resource management

As the DevOps team, we own the budget as well as the uptime. Estimates use public AWS and serverless GPU pricing (us-east-1), re-verified in week 3 and tracked against real spend every week.

Resource	Dev	Staging	Prod	Guardrail
API gateway	Local	1 container (1 vCPU / 2 GB)	1–3 containers, autoscaled	Max instance cap
Redis queue	Local	Managed Redis, pay-per-request	Managed Redis, pay-per-request	Max queue length + timeout
GPU workers	None (mock)	0 → 10 workers (mirrors prod)	0 → 10 workers	Hard cap of 10 + 5 s idle timeout
Monitoring	—	Grafana free tier	Grafana + alerts	Spend alert

How we got the GPU number

We plan for a realistic model (a ~7B LLM or mid-size vision model) at **~4 s of GPU time per request**, on a 24 GB serverless GPU at **≈ $0.70/GPU-hour**. One full replay of the traffic curve is about **15,000 requests ≈ 17 GPU-hours**. With ~20% added for cold starts and idle timeouts, that's **≈ 20 GPU-hours ≈ $14 per full run**. Runs fail in real projects, so we budget 10 staging runs.

Project budget

Item	Basis	Estimate
Infrastructure: staging + prod, live ~8 weeks		
API containers (Fargate ARM)	0.5 vCPU / 1 GB × 2 envs, minimum size between tests	$60
Load balancers	1 ALB per env	$70
Networking	Managed NAT in prod, NAT instance in staging	$70
Redis (ElastiCache t4g.micro)	1 per env	$50
Logs, registry, DNS	CloudWatch, ECR, Route 53	$20
GPU and testing		
GPU smoke tests	Bringing up workers	$30
Staging load tests	10 full runs × $14	$140
Prod rehearsals + defence demo	4 full runs × $14	$55
Always-on baseline	T4 at $0.526/hr × ~50 h	$25
Load generator	Spot VM during tests; a laptop can't reliably drive 1,000 users	$10
Contingency (~20%)	Reruns, forgotten resources, price changes	$110
Total		**≈ $640**

Month	What drives spend	Planned	Cap
October	Dev is free; staging comes up late in the month	≈ $120	$1,000
November	Both environments live, all staging load tests	≈ $400	$1,000
December	Prod rehearsals + defence, then tear-down	≈ $120	$1,000

Funding. AWS student / startup credits and serverless GPU free credits first. We ask the professor only for the gap. *Confirm eligibility in week 3.* If funding is short, the **lean plan** (tear down between test weeks, 5 runs, smaller model) costs ≈ $250.

Why this is still the cheap option. ≈ $640 buys the whole project: two environments and 14 load tests. **One** always-on T4 for the same 10 weeks costs ≈ $880 and still can't handle the spike.

Guardrails

- **AWS Budget of $1,000/month** with alerts at 50%, 80% and 100%; the cost of every load test is logged in the decisions log.
- **Kill switch:** one command sets GPU workers to 0 in any environment.
- **Worker cap of 10**, so a runaway test can't cost more than ≈ $7/hour in GPU time.

07 Team we need

What everyone does

- Writes code, reviews someone else's pull requests, and deploys through dev → staging → prod
- Takes part in load tests and logs results, costs and decisions in the shared decisions log

- Joins the weekly 30-minute sync and presents their own area at the defence

Roles

Each role has one owner who leads that area. It's not the only thing they touch.

Role	Owner	Leads
Platform / DevOps	Abhinav	Cloud setup, the three environments, CI/CD, scaling, budget
Backend	*[name]*	The API and queue: how requests come in, wait and get answered
ML / GPU	*[name]*	The model and GPU workers, including the always-on comparison
Test & Observability	*[name]*	Load tests, dashboard, and the final report and slides

With three people, Test & Observability is shared between Backend and ML. Specific tasks per person are assigned after the week 3 design and tracked week by week.

08 10-week plan

Week	Phase	Deliverable	Environment
1–2	Problem + team	Problem statement, alternatives, team, this plan	—
3	Design	Architecture diagram, tool choices with reasons, traffic assumptions confirmed, cost model	—
4–5	Minimum path	One request flows API → queue → worker → result; CI deploys to dev	Dev
6	Real model + scaling	Serverless GPU workers, autoscaling, scale-to-zero	Staging
7	Load test #1	Full traffic curve replayed; first real numbers	Staging
8	Fix + baseline	Fix what broke; always-on GPU comparison	Staging
9	Prod readiness	Promote to prod, alerts, rollback and kill-switch drill, dashboard final	Prod
10	Launch (defence)	Live demo + recorded backup, report, slides	Prod

Reading Week (Oct 10–18) falls in week 4. Align with the official Splash milestones once confirmed.

09 What we deliver (and what code is for)

Code is kept deliberately small: just enough to produce trustworthy measurements. What we are graded on is the **evidence**: the measured numbers from §03, the tradeoffs in our decisions log, and what failed and how we fixed it. We will not build user accounts, a polished frontend or a full product.

10 Risks

Risk	Response
Credits / funding fall short	Lean plan (≈ $250): tear down between test weeks, fewer runs, smaller model; extrapolate from measured cost per request
GPU cold starts too slow	Measure them; try a smaller model or one warm worker; report the tradeoff honestly
Live demo fails on the day	Recorded backup run + saved dashboards
Scope too big for 10 weeks	Minimum path by week 5; baseline comparison is the first thing cut

11 Why this problem (alternatives considered)

We also assessed real-time fraud detection, zero-downtime database migration and offline-first sync (CRDTs). We chose GenAI scaling because the team has run into it in real deployments (AWS, RunPod), it produces hard numbers we can measure, and it matches where we want our careers to go.

12 Open questions for the professor

- Our notes say Splash is "no budget". Is the ≈ $640 plan (capped at $1,000/month) acceptable, or should we run the ≈ $250 lean plan? Can the gap after credits be funded?
- What is graded at the defence: demo, report, individual contribution?
- Are there official milestones before December?

13 Simpler alternative: "The Cloud Bill Nobody Owns"

A smaller, lower-risk project, if the main one is too big or too expensive.

> **Problem statement.** Small teams waste a large share of their cloud bill on resources nobody is using: dev environments left running overnight, forgotten test machines, unattached disks, old snapshots. How can a small team find and stop that waste automatically, without breaking anything that matters?

Why it's real. Every startup that moves fast leaves resources behind: a test service from last month, a staging database nobody turned off. Nobody owns the bill, so it keeps growing. It's the "our job is to save the company money" lecture point in its simplest form.

What we build

- **Scanner:** a scheduled job (Lambda, daily) that lists resources and flags idle ones: EC2/ECS with ~0% CPU, unattached EBS volumes, old snapshots, idle load balancers, and resources missing an owner tag.
- **Scheduler:** automatically stops dev and staging environments outside working hours and starts them again in the morning.
- **Report:** a weekly email/Slack summary: what was found, what was stopped, and how many dollars were saved.
- **Safety rules:** never touches anything tagged `env=prod`; stop, never delete; everything can be undone with one command.

How we prove it works

We build a small "messy startup" account on purpose, with a few running services, a forgotten VM, orphan disks and a staging stack left on 24/7. Then we let the tool run for 2–3 weeks and compare the bill with and without it. Success means **measured savings of 30%+ on non-prod spend, with zero prod resources touched.**

	Main project (GPU scaling)	Alternative (cloud waste)
Difficulty	High: GPUs, queues, autoscaling, load tests	Moderate: AWS APIs, scheduling, reporting
Budget	≈ $640 (≈ $250 lean), cap $1,000/month	≈ $60–$100 total
Team	4 people	2–3 people (DevOps, backend, reporting/testing)
Main risk	Cost and scope	Being too simple; mitigated by measuring real savings
Demo	Live traffic spike with workers scaling up	Before/after bill plus a live "found and stopped" run